

Tutorial on PandaROOT Simulation & Analysis

PANDA Collaboration Meeting

Dec. 2013, GSI

Klaus Götzen

GSI Darmstadt



Things you can practice

- Running Simulation
 - Box generator
 - EvtGen
 - DPM
- Submitting Jobs to Prometheus
- Performing Analysis on prepared Data Sets

Basic Documentation – Rho Tutorial

- A all times up-to-date tutorial can be found here:
<http://panda-wiki.gsi.de/cgi-bin/view/Computing/PandaRootRhoTutorial>

Edit Attach Printable

[Computing.PandaRootRhoTutorial](#) r1.15 - (

Simulation and Analysis in PandaRoot with RHO (Updated: Sep. 4, 2013; Tested with rev 21585)

- ↓ Preface/Requirements
- ↓ General Documentation of Rho classes
- ↓ Files in directory tutorials/rho
- ↓ 1. Simulating the Signal Events
 - ↓ 1.1. Event Generation
 - ↓ 1.2. Simulation, Digitization, Reconstruction and Particle Identification
- ↓ 2. Analysis of Signal Events
 - ↓ 2.1. Data Access
 - ↓ 2.2. Particle Identification
 - ↓ 2.2.1. Set Mass Hypothesis only
 - ↓ 2.2.2. Various PID Criteria
 - ↓ 2.2.3. PID Algorithms
 - ↓ 2.2.4. Stand-alone PID Selector
 - ↓ 2.3. Combinatorics
 - ↓ 2.4. Monte Carlo Truth Match
 - ↓ 2.4.1. Accessing the MC truth
 - ↓ 2.4.2. Full MC truth match
 - ↓ 2.4.3. MC Truth List
 - ↓ 2.5. Fitting
 - ↓ 2.5.1. Vertex Constraint
 - ↓ 2.5.2. 4-Constraint
 - ↓ 2.5.3. Mass constraint
- ↓ 3. Analysis in a Task
 - ↓ 3.1 Create the class PndTutAnaTask
 - ↓ 3.2 Compile the class and build a library
 - ↓ 3.3 Loading library and running the macro

Preface/Requirements

This tutorial aims to demonstrate, how simulation and analysis can be performed with the [PandaRoot](#) framework. The example channel is

p pbar -> Psi(2S)-> J/Psi (-> mu+mu-) pi+ pi-

The analysis part will cover

- Data access
- Particle Identification
- Combinatorics

Basic Documentation – Rho Classes

<http://panda-wiki.gsi.de/cgi-bin/view/Computing/PandaRootAnalysisJuly13>

[Edit](#) [Attach](#) [Printable](#)

[Computing.PandaRootAnalysisJuly13](#)

Documentation of Rho Classes

Index

Here you can find information about:

- [PndAnalysis](#) - [Interface](#) - [Lists and Keys](#) - [PID Algo Names](#) - [MC Truth Match](#)
- [RhoCandidate](#) - [Interface](#)
- [RhoCandList](#) - [Interface](#)
- [Particle Selectors](#) - [Kinematic Selectors](#) - [PID Selectors](#)
- [Fitters](#) - [Vertex Fitting](#) - [4C Fitting](#) - [Kinematic Fitting](#)

A tutorial for Rho with an example simulation and analysis can be found [on this Wiki page](#).

- ↓ [Index](#)
- ↓ [1. PndAnalysis - Data Access](#)
 - ↓ [1.0 Interface of PndAnalysis](#)
 - ↓ [1.1 Initialization and Event Loop](#)
 - ↓ [1.2 Access particle lists](#)
 - ↓ [1.2.1 List Keys](#)
 - ↓ [1.2.2 PID Array Names](#)
 - ↓ [1.3 Monte Carlo Truth List](#)
 - ↓ [1.4 Monte Carlo Truth Match](#)
- ↓ [2. RhoCandidate - Handling Particles](#)
 - ↓ [2.0 Interface of RhoCandidate](#)
 - ↓ [2.1 Creating Candidates](#)
 - ↓ [2.2 Combinatorics](#)
 - ↓ [2.3 Bit Marker Concept](#)
- ↓ [3. RhoCandList - Handling Lists of Candidates](#)
 - ↓ [3.0 Interface of RhoCandList](#)
 - ↓ [3.1 Combinatorics with lists](#)
 - ↓ [3.2 Using selectors](#)
- ↓ [4. Particle Selectors](#)
 - ↓ [4.1 Kinematic Selectors](#)
 - ↓ [4.2 PID Selection](#)
 - ↓ [4.2.1 PndAnaPidCombiner - Specifying PID Algorithms](#)
 - ↓ [4.2.2 PndAnaPidSelector - PndAnalysis Style Selection](#)
 - ↓ [4.2.3 RhoSimpleSelector](#)

Basic Documentation – Job Submission

<http://panda-wiki.gsi.de/cgi-bin/view/Computing/PandaRootSimulationPrometheus>

Edit Attach Printable

[Computing.PandaRootSimulationPrometheus](#) r1.2 - 14 Nov 2013 - 16:16 - [KlausGoetzen](#) [topic end](#)

Using Prometheus@GSI for PandaROOT Simulation/Analysis

- ↓ [Preface/Requirements](#)
- ↓ [Files](#)
- ↓ [Run in local shell](#)
 - ↓ [Box Generator](#)
 - ↓ [!EvtGen](#)
 - ↓ [DPM](#)
- ↓ [Submit Jobs to Prometheus](#)
 - ↓ [Box Generator](#)
 - ↓ [!EvtGen](#)
 - ↓ [DPM](#)
- ↓ [Analyzing results](#)
 - ↓ [Local running analysis](#)
 - ↓ [Submit analysis job\(s\)](#)

Preface/Requirements

This wiki explains how to run simulation and analysis for larger amounts of events by using the [Prometheus Cluster @ GSI](#). The necessary macros/scripts for this wiki page can be found under [macro/prod](#).

All macros/scripts should be considered as examples and modified to match your particular needs.

What is needed in advance **before** being submit jobs:

- login to a lsub machine by [ssh pro.hpc.gsi.de](#)
- create a work directory on HERA, e.g. [/hera/panda/yourname/pandaroot](#)
- install PandaROOT in that place, since your GSI linux home directory is not mounted on Prometheus nodes. (SIMPAT is [/cvmfs/fairroot.gsi.de/fairsoft/apr13/](#) for apr13 external packages)
- adapt the line [./path/to/your/current/build/config.sh](#) which sources your current config.sh in the job script files ([job_prod.sge](#), [job_prod_pgun.sge](#), and [job_ana.sge](#))

Running Simulation locally

- Directory: **macro/prod**
- **Box Generator**

```
./runall_prod_pgun.sh <prefix> <N_ev> <PDG code> <max mom> <mult>
```

<prefix> - Prefix for output files
<N_ev> - Number of events to simulate
<PDG code> - PDG code of particle(s) to be created
<max mom> - Generate particles with $p = 0.01 \dots <max\ mom>$
<mult> - Multiplicity in single events

Example: ./runall_prod_pgun.sh PiPlus 1000 211 1

- **EvtGen/DPM**

```
./runall_prod.sh <prefix> <N_ev> <decay> <pbar mom> <res>
```

<prefix> - Prefix for output files
<N_ev> - Number of events to simulate
<decay> - Decay file name or keyword 'DPM'
<pbar mom> - Anti-proton beam momentum
<res> - Initial resonance for EvtGen

Example 1: ./runall_prod.sh D0Kpi 1000 D0toKpi.dec 10.0 pbarpSystem

Example 2: ./runall_prod.sh Dpm10GeV 1000 DPM 10.0

Submitting Simulation Jobs to Prometheus@GSI

- Directory: `macro/prod`
- Wiki page:
<http://panda-wiki.gsi.de/cgi-bin/view/Computing/PandaRootSimulationPrometheus>
- Prerequisites
 - Login to a lxsub machine by `ssh pro.hpc.gsi.de`
 - Create a work directory on HERA (e.g. `/hera/panda/yourname/mydir`)
 - Install PandaROOT in that place
(`SIMPATH = /cvmfs/fairroot.gsi.de/fairsoft/apr13/`
for apr13 external packages)
 - Adapt the line `. /path/to/your/current/build/config.sh` in `job_prod.sge`,
`job_prod_pgun.sge`, and `job_ana.sge`

Submitting Simulation Jobs to Prometheus@GSI

- Directory: **macro/prod**
- **Box Generator**

```
qsub -t <min>-<max> job_prod_pgun.sge <prefix> <N_ev> <PDG code> <max mom> <mult>
```

<prefix> - Prefix for output files
<N_ev> - Number of events to simulate
<PDG code> - PDG code of particle(s) to be created
<max mom> - Generate particles with $p = 0.01 \dots <max\ mom>$
<mult> - Multiplicity in single events

Example: `qsub -t 1-20 job_prod_pgun.sge PiPlus 1000 211 1`

- **EvtGen/DPM**

```
qsub -t <min>-<max> job_prod.sge <prefix> <N_ev> <decay> <pbar mom> <res>
```

<prefix> - Prefix for output files
<N_ev> - Number of events to simulate
<decay> - Decay file name or keyword 'DPM'
<pbar mom> - Anti-proton beam momentum
<res> - Initial resonance for EvtGen

Example 1: `qsub -t 1-20 job_prod.sge D0Kpi 1000 D0toKpi.dec 10.0 pbarpSystem`

Example 2: `qsub -t 1-20 job_prod.sge Dpm10GeV 1000 DPM 10.0`

Analysis

- Directory: `tutorials/dec13`
- Example data: `/hera/panda/tutdata/`
- Description: `/hera/panda/tutdata/README`
- Modify `tut_ana.C` for your analysis!
- Template macro (multiple file input):
`tut_ana.C`

```
tut_ana.C("<prefix>", <min>, <max>, <nevt>)
```

<code><prefix></code>	- Prefix for input files (Pattern: <code><prefix>_<num>_pid.root</code>)
<code><min></code>	- Minimum number of file to be processed
<code><max></code>	- Maximum number of file to be processed
<code><nevt></code>	- Number of events to be processed

Example: `root -l 'tut_ana.C("D0data",1,10)'`

Processes: `D0data_1_pid.root, ..., D0data_10_pid.root`

Analysis Code in a Task

AnalysisMacro.C

```
void AnalysisMacro()
{
    FAIRROOT Initialization

    HISTOGRAM Declaration
    HISTOGRAM Initialization

    GLOBALS Declaration
    GLOBALS Initialization

    while ( fAna->GetEvent() )
    {
        ANALYSIS CODE
    }

    SAVING HISTOGRAMS
}

ROUTINES Definition
ROUTINES Implementation
```

AnalysisMacroTask.C

```
void AnalysisMacroTask()
{
    FAIRROOT Initialization
    PndMyAnaTask *mytask = new PndMyAnaTask()
    fRunAna->AddTask(mytask);
}
```

PndMyAnaTask.h

```
class PndMyAnaTask : public FairTask
{
private:
    GLOBALS Declaration
    HISTOGRAM Declaration
    ROUTINES Definition
};
```

PndMyAnaTask.cxx

```
ROUTINES Implementation

PndMyAnaTask::Init()
HISTOGRAM/GLOBALS Initialization

PndMyAnaTask::Exec()
ANALYSIS CODE

PndMyAnaTask::Finish()
SAVING HISTOGRAMS
```

Analysis Code in a Task

- Directory: `tutorials/dec13`
- Example data: `/hera/panda/tutdata/`
- Modify `PndTutAnaTask.cxx,h` for your analysis!
- Compile with

```
cd $FAIRLIBDIR/../../tutorials/dec13  
make -j8
```

- Templates (multiple file input):
`PndTutAnaTask.cxx`, `PndTutAnaTask.h`, `tut_ana_task.C`

```
tut_ana_task.C("<prefix>", <min>, <max>, <nevt>)
```

<code><prefix></code>	- Prefix for input files (Pattern: <code><prefix>_<num>_pid.root</code>)
<code><min></code>	- Minimum number of file to be processed
<code><max></code>	- Maximum number of file to be processed
<code><nevt></code>	- Number of events to be processed

Example: `root -l 'tut_ana_task.C("D0data",1,10)'`

Processes: `D0data_1_pid.root, ..., D0data_10_pid.root`

Working with RhoTuple

- Instead of Histograms → N-Tuple structure to store results
- RhoTuple → On-the-fly creation/filling of branches

```
RhoTuple *ntp = new RhoTuple("ntp","my analysis results");
```

- Set branch value & create branch if not available

```
ntp->Column(TString "<name>", Type_t <var>, Type_t <default>)
```

<name> - Branch name

<var> - Variable to be filled in branch (needs strict casting!)

<default> - Default value for branch, if not filled explicitly for one event

- Fill values to internal tree

```
ntp->DumpData();
```

RhoTuple Example

- Usage of **RhoTuple** could look like:

```
RhoTuple *ntp = new RhoTuple("ntp","J/psi analysis");

...
// ... in event loop ...
for ( j=0 ; j<jpsi.GetLength() ; ++j )
{
    RhoCandidate      *mup  = jpsi[j]->Daughter(0);
    PndPidCandidate *mupr = (PndPidCandidate*) mup->GetRecoCandidate();

    // *** store event number and candidate number in current event
    ntp->Column( "ev",      (Float_t) evnumber );
    ntp->Column( "cand",    (Float_t) j );

    // *** basic information about J/psi
    ntp->Column( "jpsim",    (Float_t) jpsi[j]->M() );
    ntp->Column( "jpsip",    (Float_t) jpsi[j]->P() );

    // *** info from daughters
    ntp->Column("mupp",      (Float_t) mup->P() );
    ntp->Column("mupiron",  (Float_t) mupr->GetMuoIron() );
    ...
}
```

Analysis Approach Proposal

How to get started?

- Select channel you want to (simulate and) analyse
- Outside event loop
 - Prepare RhoCandLists for final states and composites
 - Prepare selectors and certain things you need (beam 4-vector etc)
- Inside event loop
 - Fill lists of final states (e, mu,..., gammas) with certain PID algo
 - Pre selection on lists (energy/momentum/PID/...)
 - Do combinatorics to form composites
 - Rough selection before further combinatorics recommended
- Good quantities to take a look to/cut on/store to n-tuple:
 - Final states: E_{photon} / PID info / p_{trk} / P4 (lab/cms) / ...
 - Composites: m_{inv} / p / p_{cms} / m_{miss} / vtx info / P4 (lab/cms) / ...

And now ... **let's go!!**

Backup about Interfaces, PID, McTruth, Fitting

Data Access in PandaROOT

- PandaROOT object: **PndAnalysis**

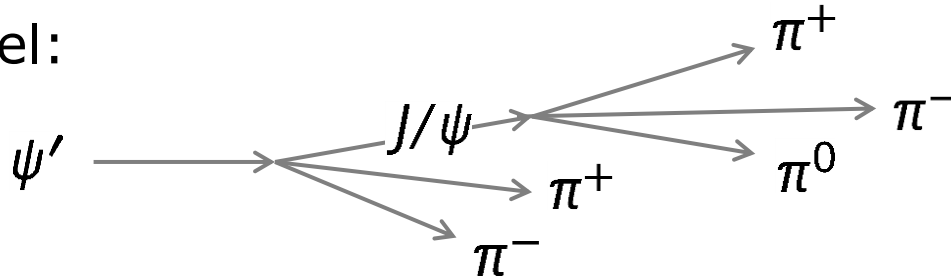
```
FairRunAna *fRun = new FairRunAna();
fRun->SetInputFile("pid_complete.root");

PndAnalysis *ana = new PndAnalysis();
RhoCandList eplus, eminus, muplus, muminus, mct;
TString myPidAlgosElectron = "PidAlgoEmcBayes;PidAlgoDrc"
TString myPidAlgosMuon     = "PidAlgoMdtHardCuts"
...
while ( ana->GetEvent() )
{
    ana->FillList( eplus,    "ElectronTightPlus",  myPidAlgosElectron );
    ana->FillList( eminus,   "ElectronTightMinus", myPidAlgosElectron );
    ana->FillList( muplus,    "MuonTightPlus",      myPidAlgosMuon );
    ana->FillList( muminus,   "MuonTightMinus",     myPidAlgosMuon );
    ana->FillList( mct,       "McTruth" );
    ...
}
```

- Features:
 - Simple access to reco candidates and McTruth objects
 - Various PID algorithms directly accessible

Combinatorics in PandaROOT

- PandaROOT objects: **RhoCandList & RhoCandidate**
- Example channel:



```
RhoCandList piplus, piminus, gamma, pi0, jpsi, psip; // define RhoCandList's

pndana->FillList(piplus, "PionAllPlus"); // positive chrg trks with pion hypo
pndana->FillList(piminus, "PionAllMinus"); // negative chrg trks with pion hypo

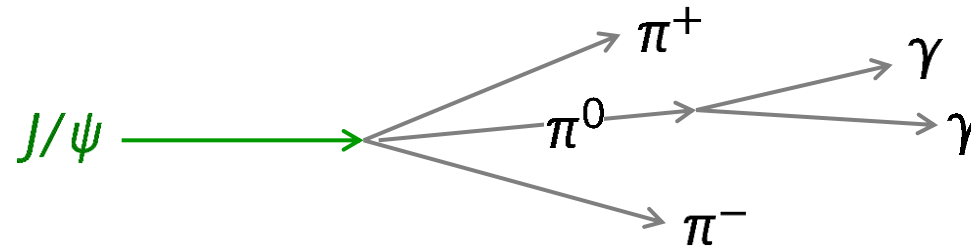
pndana->FillList(gamma, "Neutral"); // neutral particle candidates

pi0.Combine(gamma, gamma); // cares about double counting
jpsi.Combine(piplus, piminus, pi0);
psip.Combine(jpsi, piplus, piminus); // cares about overlaps

for (int j=0; j<psip.GetLength(); ++j) // loop over candidates in TCandList
{
    masshisto.Fill( psip[j]->M() ); // and fill e.g. a mass histo
}
```

Access to Genealogy

- PandaROOT object: **RhoCandidate**
- Example channel:



```
pi0.Combine(gamma, gamma);           // create pi0 candidates
jpsi.Combine(piplus, piminus, pi0);   // create J/psi candidates
...
cout << jpsi[i]->NDaughters() << endl; // prints number of daughters (3, not 4!)

RhoCandidate *pip = jpsi[i]->Daughter(0); // the pi+ candidate
RhoCandidate *pim = jpsi[i]->Daughter(1); // the pi- candidate
RhoCandidate *pi0 = jpsi[i]->Daughter(2); // the pi0 candidate (composite!)

RhoCandidate *gam1 = pi0->Daughter(0); // the 1st gamma
RhoCandidate *gam2 = pi0->Daughter(1); // the 2nd gamma
```

Particle Candidate in PandaROOT

- PandaROOT object: **RhoCandidate**
- Some important accessors:

Return type	Method	Information
TLorentzVector	P4()	4-vector
TVector3	P3()	Momentum vector
TVector3	Pos()	Origin
Double_t	Charge()	Charge
Double_t	M(), P(), E()	Mass, Momentum, Energy
Int_t	NDaughters()	Number of daughters
RhoCandidate*	Daughter(i)	i-th daughter in decay tree
RhoCandidate*	TheMother()	Mother in decay tree
RhoCandidate*	Combine(TCandidate &c,...)	Creates composite candidate
Bool_t	Overlaps(TCandidate &c)	Checks for collision in tree
RhoCandidate*	GetMcTruth()	MC truth object
Int_t	PdgCode()	PDG code
FairRecoCandidate*	GetRecoCandidate()	Access to reco information

Micro Candidate in PandaROOT

- PandaROOT object: **PndPidCandidate** (*FairRecoCandidate*)
- Some accessors:

Return type	Method	Information
Int_t	GetMvdHits()	Number of MVD hits
Float_t	GetMvdDEDX()	dE/dx measurm. from MVD
Int_t	GetSttHits()	Number of STT hits
Float_t	GetSttMeanDEDX()	dE/dx measurm. from STT
Float_t	GetDrcThetaC()	θ_c measurement from DIRC
Int_t	GetDrcNumberOfPhotons()	Number Photons in DIRC
Float_t	GetEmcCalEnergy()	Calibrated EMC cluster energy
Int_t	GetEmcNumberOfCrystals()	Number of crystals in cluster
Int_t	GetMuoNumberOfLayers()	Number of layers hit in MUO det
Float_t	GetMuoProbability()	Probability for being a muon

Particle List in PandaROOT

- PandaROOT object: **RhoCandList**
- Some important accessors:

Method	Information
<code>Int_t GetLength()</code>	number of particles in list
<code>void Add(RhoCandidate*)</code>	Appends candidate to list
<code>void Remove(RhoCandidate*)</code>	Removes candidate from list
<code>RhoCandidate* operator[] (Int_t)</code>	Returns a candidate
<code>void Append(RhoCandList&)</code>	Appends candidates from another list
<code>void Combine(RhoCandList&,...)</code>	Combinatorics for up to five TCandLists
<code>void Select(VAbsPidSelector*)</code>	Selects (=modifies) list
<code>void Select(RhoCandList &, VAbsPid...)</code>	Selects from another list
<code>void Cleanup()</code>	Empties list

Combinatorics in PandaROOT (by hand)

- Example channel: $J/\psi \rightarrow \pi^+ \pi^- \pi^0 (\rightarrow \gamma\gamma)$

```
RhoCandList piplus, piminus, gamma, pi0, jpsi;           // define TCandList's
...
pi0.Cleanup(); jpsi.Cleanup();                          // clean lists !!

for (int i=0; i<gamma.GetLength()-1; i++) {             // nested loops
    for (int j=i+1; j<gamma.GetLength(); j++) {
        if (gamma[i]->E()>Emin || gamma[j]->E()>Emin) { // do e.g. some check
            RhoCandidate *comb = gamma[i]->Combine(gamma[j]); // create comb. candidate
            pi0.Add(comb);                                     // add it to list
        }
    }
}

for (int i=0; i<piplus.GetLength(); i++) {              // and the same for
    for (int j=0; j<piminus.GetLength(); j++) {          // the J/psi candidates
        for (int k=0; k<pi0.GetLength(); k++) {
            RhoCandidate *comb = piplus[i]->Combine(piminus[j], pi0[k]);
            jpsi.Add(comb);
        }
    }
}
...
```

Combinatorics in PandaROOT (by hand)

- Example channel (overlap check): $D_s^\pm \rightarrow \phi \pi^\pm \rightarrow K^+ K^- \pi^\pm$

```
RhoCandList pipm, kplus, kminus, ds; // define RhoCandList's
...
ds.Cleanup(); // clean list

for (int i=0; i<kplus.GetLength(); i++) // nested loops
{
    for (int j=0; j<kminus.GetLength(); j++)
    {
        RhoCandidate *phi = kplus[i]->Combine(kminus[j]); // create phi candidate

        for (int k=0; k<pipm.GetLength(); k++)
        {
            if (!pipm[k]->Overlaps(phi)) // check for overlap
            {
                RhoCandidate *comb = phi->Combine(pipm[k]);
                ds.Add(comb);
            }
        }
    }
}
...
```


PID Concept in PandaROOT

- PandaROOT objects:

PndAnalysis, PndAnaPidCombiner, PndAnaPidSelector

```
PndAnalysis *pndana= new PndAnalysis();  
pndana->FillList(eplus, "ElectronLoosePlus", "PidAlgoEmcBayes;PidAlgoMvd");  
pndana->FillList(eminus, "ElectronLooseMinus", "PidAlgoEmcBayes;PidAlgoMvd");
```

Or ,by hand`:

```
RhoCandList charged, kaonLoose;  
  
PndAnaPidSelector kaonSel("KaonSelector");  
kaonSel.SetSelection("KaonLoose");           // set selection criterion  
  
PndAnaPidCombiner pidComb("PidCombiner");  
pidComb.SetTcaNames("PidAlgoDrc;PidAlgoMvd"); // set algo's  
  
while (evr->GetEvent())  
{  
    pndana->FillList(charged, "Charged");      // start w/ charged candidates  
  
    pidComb.Apply(charged);                    // copy probab. to candidates  
    kaonSelector.Select(charged, kaonLoose);  // select kaons from charged  
}
```

MC Truth Access

- For final state particles via `RhoCandidate::GetMcTruth`

```
ana->FillList( eplus, "ElectronTightPlus", "PidAlgoEmcBayes" );  
RhoCandidate *truth = eplus[0]->GetMcTruth();
```

- The MC truth particles have the complete genealogy

```
if ( truth != 0x0 ) {  
    RhoCandidate *mother = truth->TheMother();  
    if ( truth->NDaughters() > 1 ) {  
        RhoCandidate *firstDaughter = truth->Daughter(0);  
        RhoCandidate *secondDaughter = truth->Daughter(1);  
    }  
}
```

- Also complete MC truth list available

```
ana->FillList( mct, "McTruth" );  
...  
RhoCandidate *firstDaughter = mct[0]->Daughter(0);
```

Usage of PndAnalysis::McTruthMatch

- For matching, **composite** candidates have to have **type set**

```
PndAnalysis *ana = new PndAnalysis();

while ( ana->GetEvent() )
{
    ana->FillList(muplus, "MuonPlus");
    ...
    jpsi.Combine(muplus, muminus);
    jpsi.SetType( "J/psi" );      // set type for J/psi (names like TDatabasePDG)

    psi2s.Combine(jpsi, piplus, piminus);
    psi2s.SetType( "psi(2S)" );  // set type for psi(2S)

    bool match  = ana->McTruthMatch( psi2s[0] ); // match for RhoCandidate

    int nmatch  = ana->McTruthMatch( psi2s );    // match complete RhoCandList

    for (int j=0; j<psi2s.GetLength(); ++j)
    {
        RhoCandidate *truth = psi2s[j].GetMcTruth(); // access truth of composites
        ...
    }
}
```

Fitting in PandaROOT

- Fitters basically all have a similar interface
- Fit results are **attached to RhoCandidates**
- Can be accessed as full tree, **allows cascaded fitting!**
- E.g. vertex fitting + mass fitting might look like this:

```
RhoCandidate *lambda = pplus->Combine( pminus );
TVector3 IP( 0,0,0 );
...
PndKinVtxFitter fitvtx( lambda );           // setup vertex fitter
fitvtx.AddPointingConstraint( IP );         // add pointing constraint
fitvtx.FitAll();                           // perform fit

RhoCandidate *lambda_vtx = lambda->GetFit(); // access fit results

PndKinFitter fitmass( lambda_vtx );         // setup mass fitter
fitmass.SetMassConstraint( 1.115 );         // set mass constraint
fitmass.Fit();                             // perform fit

RhoCandidate *lambda_mass = lambda_vtx->GetFit(); // access cascaded fit results

RhoCandidate *fit_pplus  = lambda_mass->Daughter( 0 );
RhoCandidate *fit_pminus = lambda_mass->Daughter( 1 );
```

Fitting Example: Vertex-Fit

- PandaROOT object: **PndKinVtxFitter**

```
// ... in event loop ...

for (j=0; j<jpsi.GetLength(); ++j)
{
    PndKinVtxFitter vtxfitter(jpsi[j]);           // instantiate vertex fitter
    vtxfitter.Fit();                             // perform fit

    RhoCandidate *jfit = jpsi[j]->GetFit();       // get fitted candidate

    TVector3 jVtx = jfit->Pos();                  // and the vertex position
    double chi2_vtx = vtxfitter.GlobalChi2();     // and the chi^2 of the fit

    if ( chi2_vtx<max_chi )                      // if chi2 is good enough
    {                                             // fill some histos
        hjpsim_vf->Fill( jfit->M() );
        hvpos->Fill( jVtx.X(),jVtx.Y() );
    }
}
```

Fitting Example: 4C-Fit

- PandaROOT object: **PndKinFitter**

```
// the lorentz vector of the initial psi(2S); important for the 4C-fit
TLorentzVector ini(0, 0, 6.231552, 7.240065);

// ... in event loop ...
for (j=0;j<psi2s.GetLength();++j)
{
    PndKinFitter fitter(psi2s[j]);           // instantiate the kinematic fitter
    fitter.Add4MomConstraint(ini);           // set 4 vector constraint
    fitter.Fit();                           // perform fit

    RhoCandidate *jfit = psi2s[j]->Daughter(0)->GetFit(); // get fitted J/psi

    Double_t chi2 = fitter.GetChi2();        // and the chi^2 of the fit

    if ( 0 != jfit )
    {
        hjpsim_4cf->Fill( jfit->M() );      // fill histogram
    }
}
```